

CS 301

2025–2026 Spring

Project Report

Group 15

Group Members:

Musab Ahmed Khan 33017

Imran Hasanzade 33537

Umar Hanif Abdul Aziz 32758

Contents

1	Problem Description	3
1.1	Overview	3
1.2	Decision Problem	3
1.3	Optimization Problem	3
1.4	Example Illustration	4
1.5	Real-World Applications	4
1.6	Hardness of the Problem	5
1.6.1	Proving NP	5
1.6.2	Proving NP-Hard	5
1.6.3	Proving NP-Complete	5
2	Algorithm Description	6
2.1	Brute Force Algorithm	6
2.1.1	Overview	6
2.1.2	Pseudocode	6
2.2	Heuristic Algorithm	8
3	Algorithm Analysis	8
3.1	Brute Force Algorithm	8
3.1.1	Correctness Analysis	8
3.1.2	Time Complexity Analysis	8
3.1.3	Space Complexity Analysis	9
3.2	Heuristic Algorithm	9
4	Sample Generation (Random Instance Generator)	9
4.1	Explanation	9
4.2	Pseudocode	10
5	Algorithm Implementations	11
5.1	Brute Force Algorithm	11
5.2	Heuristic Algorithm	12
6	Experimental Analysis of the Performance	13
7	Experimental Analysis of the Quality	13
8	Experimental Analysis of the Correctness	13
9	Discussion	13

References**14**

1 Problem Description

Name	Subset Sum
Input	A finite set $S = \{s_1, s_2, \dots, s_n\}$ of integers and a target integer k
Question	Does there exist a subset $T \subseteq S$ such that $\sum_{s \in T} s = k$?

Table 1: Description of the Subset Sum Problem

1.1 Overview

The Subset Sum problem is one of the most studied problems in computer science. Given a set of integers and a target value, the question is whether some subset of those integers adds up to exactly that target. In the optimization version, the goal is to find the subset whose sum is as large as possible without exceeding the target.

The problem is easy to state, but hard to solve in general. No known algorithm handles all instances efficiently. It has been proven to be NP-complete, meaning it belongs to a class of problems for which no polynomial-time algorithm is expected to exist unless $P = NP$.

1.2 Decision Problem

The decision version asks a yes-or-no question:

Given a finite set $S = \{s_1, s_2, \dots, s_n\}$ of integers and a target integer k , does there exist a subset $T \subseteq S$ such that $\sum_{s \in T} s = k$?

Formally:

$$\text{SUBSET-SUM} = \{ \langle S, k \rangle \mid \exists T \subseteq S \text{ such that } \sum_{s \in T} s = k \}$$

For example, given $S = \{3, 1, 4, 2, 6\}$ and $k = 7$, the answer is YES because $\{1, 6\}$ sums to 7. If $k = 15$, the answer is NO because the total sum of all elements is only 16, and no proper subset reaches 15.

1.3 Optimization Problem

The optimization version does not ask whether a solution exists. It asks for the best feasible one:

Given a finite set $S = \{s_1, \dots, s_n\}$ of non-negative integers and a target k , find a subset $T \subseteq S$ that maximizes $\sum_{s \in T} s$ subject to $\sum_{s \in T} s \leq k$.

If a subset summing to exactly k exists, that is trivially optimal. Otherwise the goal is to get as close to k as possible from below.

Note: in the experimental sections of this report, only the optimization version is tested. For the decision version, the test is simply whether the algorithm returns a subset summing to exactly k .

1.4 Example Illustration

Example 1 (Decision: YES). Let $S = \{4, 11, 16, 21, 27\}$ and $k = 25$. The subset $\{4, 21\}$ sums to 25. The decision answer is YES.

Example 2 (Decision: NO; Optimization returns near-optimal). Let $S = \{5, 7, 12\}$ and $k = 10$. No subset sums to exactly 10. The feasible subsets (those with sum ≤ 10) and their sums are:

Subset	Sum
$\{\}$	0
$\{5\}$	5
$\{7\}$	7

The decision answer is NO. The optimization answer is $\{7\}$ with sum 7.

Example 3 (k equals sum of all elements). Let $S = \{4, 6, 10\}$ and $k = 20$. The full set $\{4, 6, 10\}$ sums to 20. Both the decision and optimization answers return the full set.

Example 4 (All elements exceed k). Let $S = \{10, 20, 30\}$ and $k = 5$. No non-empty subset is feasible. The optimization answer is the empty set with sum 0.

1.5 Real-World Applications

Resource allocation and budgeting. A direct application is deciding whether a collection of tasks or items with associated costs can exactly fill a given budget. For instance, selecting jobs from a list so that their total processing time exactly matches an available time slot is an instance of Subset Sum.

Cryptography. The Merkle-Hellman knapsack cryptosystem, one of the earliest public-key schemes, is built directly on the hardness of Subset Sum [4]. Encryption maps a message to a subset sum of a public key; decryption uses a hidden trapdoor that converts the hard instance into an easy one.

Load balancing. Distributing a set of jobs across processors so that each processor carries the same total load is equivalent to a partition problem, which is a special case of Subset Sum.

Finance and auditing. Identifying whether a set of transactions can exactly account for a given total is a direct use of the decision version, and appears in reconciliation and audit workflows.

1.6 Hardness of the Problem

Theorem 1.1. *The decision version of the Subset Sum problem is NP-complete.*

The proof has two parts.

1.6.1 Proving NP

A problem is in NP if a proposed solution can be verified in polynomial time. For Subset Sum, the certificate is the subset T itself. Given T , we verify it by computing $\sum_{s \in T} s$ and checking whether it equals k . This takes $O(n)$ time. Therefore, Subset Sum $\in$ NP [2] (Theorem 7.25, p. 297).

1.6.2 Proving NP-Hard

Karp [1] proved that Subset Sum is NP-hard by showing a polynomial-time reduction from the 3-SAT problem. Since 3-SAT is NP-complete and every NP problem reduces to it in polynomial time, a polynomial-time reduction from 3-SAT to Subset Sum implies that Subset Sum is at least as hard as any problem in NP. The complete reduction is given in [2] (Section 7.5, p. 319) and in the original paper [1]. We cite these rather than reproduce the proof here.

1.6.3 Proving NP-Complete

Since Subset Sum is both in NP and NP-hard, it is NP-complete. This means that unless $P = NP$, no polynomial-time algorithm correctly solves all instances. The brute-force algorithm in Section 2.1 therefore has exponential worst-case time, and the heuristic in Section 2.2 trades optimality for efficiency.

2 Algorithm Description

2.1 Brute Force Algorithm

2.1.1 Overview

The brute force algorithm solves the Subset Sum problem by checking every possible subset of S . For a set of n elements there are 2^n subsets in total, including the empty set. The algorithm evaluates each one and either checks for an exact match (decision version) or tracks the best feasible sum found so far (optimization version).

The implementation uses bitmask enumeration. Each integer in the range $[0, 2^n - 1]$ is treated as a binary string of length n , where bit i indicates whether element $S[i]$ is included. This lets all 2^n subsets be generated with a single loop.

This is pure exhaustive search. It does not use divide-and-conquer or dynamic programming. It is guaranteed to find the correct answer because it examines every candidate without exception. The trade-off is that its runtime grows exponentially with n , making it impractical beyond roughly $n = 25$. Its main role in this project is to provide a correct reference solution for the quality-testing experiments in Section 7.

2.1.2 Pseudocode

Algorithm 1 gives the optimization version. The decision version follows the same structure but returns as soon as a subset summing to exactly k is found.

Algorithm 1 Brute Force Optimization

Input: Set $S = [s_0, \dots, s_{n-1}]$, target k **Output:** Subset of S with maximum sum $\leq k$

```

1:  $best\_sum \leftarrow 0$ 
2:  $best\_subset \leftarrow []$ 
3: for  $mask \leftarrow 0$  to  $2^n - 1$  do
4:    $current\_sum \leftarrow 0$ 
5:    $current\_subset \leftarrow []$ 
6:   for  $i \leftarrow 0$  to  $n - 1$  do
7:     if bit  $i$  of  $mask$  is set then
8:        $current\_sum \leftarrow current\_sum + S[i]$ 
9:        $current\_subset.append(S[i])$ 
10:    end if
11:  end for
12:  if  $current\_sum \leq k$  and  $current\_sum > best\_sum$  then
13:     $best\_sum \leftarrow current\_sum$ 
14:     $best\_subset \leftarrow current\_subset$ 
15:  end if
16:  if  $best\_sum = k$  then
17:    break ▷ exact match found, no improvement possible
18:  end if
19: end for
20: return  $best\_subset, best\_sum$ 

```

Algorithm 2 Brute Force Decision

Input: Set $S = [s_0, \dots, s_{n-1}]$, target k **Output:** (FOUND, subset) if a subset sums to k , else (NOT FOUND, [])

```

1: for  $mask \leftarrow 0$  to  $2^n - 1$  do
2:    $current\_sum \leftarrow 0$ ;  $current\_subset \leftarrow []$ 
3:   for  $i \leftarrow 0$  to  $n - 1$  do
4:     if bit  $i$  of  $mask$  is set then
5:        $current\_sum \leftarrow current\_sum + S[i]$ 
6:        $current\_subset.append(S[i])$ 
7:     end if
8:   end for
9:   if  $current\_sum = k$  then
10:    return (FOUND,  $current\_subset$ )
11:   end if
12: end for
13: return (NOT FOUND, [])

```

The early exit at line 16 of Algorithm 1 is a minor optimization: once an exact match is found, the result is already optimal and no further subsets need to be checked. Without it, the algorithm still produces the correct answer but always runs for all 2^n iterations.

Design technique. Pure exhaustive search. No divide-and-conquer or dynamic programming structure is used.

2.2 Heuristic Algorithm

[This section is authored by Person 2.]

3 Algorithm Analysis

3.1 Brute Force Algorithm

3.1.1 Correctness Analysis

Theorem 3.1. *Algorithm 1 correctly solves the Subset Sum optimization problem. That is, it returns a subset $T^* \subseteq S$ such that $\sum_{s \in T^*} s \leq k$ and this sum is maximum over all subsets of S with sum at most k .*

Proof. The integers $0, 1, \dots, 2^n - 1$ are in one-to-one correspondence with all subsets of S : for any mask m , the set of indices $\{i : \text{bit } i \text{ of } m \text{ is set}\}$ uniquely identifies a subset, and every subset of S corresponds to exactly one mask. Therefore the outer loop visits every subset of S exactly once.

For each visited subset, lines 12–14 update *best_sum* when the current subset is feasible (sum $\leq k$) and strictly better than the previous best. Since every subset is visited, the optimal feasible subset is visited at some iteration. At that iteration, the condition on line 12 holds (the optimal sum is $\leq k$ by definition and is at least as large as any prior feasible sum), so *best_subset* is updated. By the end of the loop, *best_subset* holds the optimal subset.

The early exit on lines 16–17 does not affect correctness: once *best_sum* = k , no feasible subset can have a larger sum, so stopping early yields the same result. $\square$

3.1.2 Time Complexity Analysis

The outer loop runs for all integers from 0 to $2^n - 1$, giving 2^n iterations. For each iteration, the inner loop on lines 6–11 runs n times (one bit check per element), and each step takes $O(1)$ time. The total operation count is:

$$T(n) = 2^n \cdot n$$

This is both an upper and a lower bound in the worst case (when no exact solution exists and no early exit is triggered). Therefore the worst-case time complexity is:

$$T(n) = \Theta(n \cdot 2^n)$$

This is exponential in n . For $n = 20$ this is approximately 20×10^6 operations; for $n = 30$ it is around 30×10^9 , which takes seconds on typical hardware.

3.1.3 Space Complexity Analysis

The algorithm uses the following memory beyond the input array S :

- *current_subset* and *best_subset*: each stores at most n elements, so $O(n)$ each.
- Scalar variables (*mask*, *i*, *current_sum*, *best_sum*): $O(1)$.
- No recursion is used, so there is no call-stack overhead beyond $O(1)$.

Total auxiliary space (not counting the input):

$$O(n)$$

3.2 Heuristic Algorithm

[This section is authored by Person 2.]

4 Sample Generation (Random Instance Generator)

4.1 Explanation

To test the algorithms, we need a way to generate random Subset Sum instances in a controlled, reproducible manner. The generator is parametric in n , the size of the input set. Given n , it produces a set S of n random integers and a target k .

Two modes are supported:

- **Guaranteed-solution mode.** A random non-empty subset of S is chosen, and k is set to its sum. This guarantees that at least one exact solution exists. This mode is used for initial testing and for quality testing in Section 7, where both algorithms must run on instances with a known correct answer.
- **Random-target mode.** k is chosen uniformly at random from $[1, \sum S]$. Whether an exact solution exists is not predetermined. This mode produces a more realistic distribution of instances.

A `seed` parameter is available for reproducibility: the same seed always produces the same instance.

4.2 Pseudocode

Algorithm 3 Generate Instance

Input: n , value range $[\ell, h]$, *guarantee_solution* flag, *seed*

Output: A `SubsetSumInstance` (S, k)

```

1: if seed  $\neq$  NONE then
2:   initialise random generator with seed
3: end if
4:  $S \leftarrow []$ 
5: for  $i \leftarrow 1$  to  $n$  do
6:    $S[i] \leftarrow$  random integer in  $[\ell, h]$ 
7: end for
8:  $total \leftarrow \sum S$ 
9: if  $total = 0$  then
10:  return ( $S, k=0$ ) ▷ edge case: all-zero set
11: end if
12: if guarantee_solution then
13:   $r \leftarrow$  random integer in  $[1, n]$ 
14:   $indices \leftarrow$  random subset of  $\{0, \dots, n-1\}$  of size  $r$ 
15:   $k \leftarrow \sum_{i \in indices} S[i]$ 
16: else
17:   $k \leftarrow$  random integer in  $[1, total]$ 
18: end if
19: return ( $S, k$ )

```

Algorithm 4 Generate Batch

Input: List of sizes *sizes*, *reps_per_size*, value range, *guarantee_solution*, *base_seed*

Output: List of `SubsetSumInstance` objects

```

1:  $instances \leftarrow []$ ;  $seed \leftarrow base\_seed$ 
2: for each  $n$  in sizes do
3:   for  $rep \leftarrow 1$  to reps_per_size do
4:      $inst \leftarrow$  GenerateInstance( $n, \dots, seed$ )
5:      $instances.append(inst)$ 
6:      $seed \leftarrow seed + 1$  ▷ increment so each instance is different but reproducible
7:   end for
8: end for
9: return instances

```

5 Algorithm Implementations

5.1 Brute Force Algorithm

The brute force algorithm was implemented in Python as two functions: `brute_force_decision` and `brute_force_optimization`. Both accept a `SubsetSumInstance` object and return a `SubsetSumResult` containing the selected subset, its sum, a flag indicating whether the sum equals k exactly, and the wall-clock runtime. The full source code is in the project notebook (`subset_sum_analysis.ipynb`).

The implementation was tested on 16 instances and 6 edge cases, as described below.

Initial Test of the Algorithm

We ran the brute force optimization function on 16 instances across sizes $n \in \{5, 8, 10, 12, 14, 16, 18, 20\}$. The instances were split evenly: 8 with a guaranteed exact solution (seeds 100–107) and 8 with a randomly chosen target (seeds 200–207). The value range was $[1, 50]$.

#	n	k	Opt. sum	Exact	Time (s)	Mode
1	5	102	102	Yes	0.000013	guaranteed
2	8	104	104	Yes	0.000013	guaranteed
3	10	198	198	Yes	0.000118	guaranteed
4	12	223	223	Yes	0.000184	guaranteed
5	14	15	15	Yes	0.000003	guaranteed
6	16	115	115	Yes	0.000077	guaranteed
7	18	223	223	Yes	0.001379	guaranteed
8	20	295	295	Yes	0.000392	guaranteed
9	5	19	19	Yes	0.000008	random k
10	8	180	171	No	0.000141	random k
11	10	220	219	No	0.000573	random k
12	12	12	11	No	0.002807	random k
13	14	38	38	Yes	0.000004	random k
14	16	65	65	Yes	0.000054	random k
15	18	125	125	Yes	0.000315	random k
16	20	167	167	Yes	0.000035	random k

Table 2: Initial testing results for `brute_force_optimization`. All 16 instances passed the validity check. Value range $[1, 50]$, seeds 100–107 (guaranteed) and 200–207 (random k).

Summary:

- Instances tested: 16
- All passed the validity check: **16/16**
- Exact matches found: **13/16** (all 8 guaranteed + 5 of 8 random)
- Mean runtime: 0.000382 s
- Maximum runtime: 0.002807 s (instance 12, $n = 12$, random k)

All 8 guaranteed instances returned an exact match. The 5 out of 8 exact matches in the random-target group are expected: when k is chosen at random there is no guarantee an exact solution exists, and for roughly half of instances in this range, one happens to exist.

The runtime grows visibly with n . At $n = 20$ the observed time is about 3.3 ms, consistent with $2^{20} \approx 10^6$ iterations. Doubling n to 40 would require $2^{40} \approx 10^{12}$ iterations, making the algorithm infeasible beyond roughly $n = 25$ to 30 on standard hardware.

No failures were found during testing. No fixes were needed.

Edge Case Tests

Table 3 shows results for boundary inputs.

Case	n	k	Returned sum	Exact	Valid	Time (s)
Empty set	0	5	0	No	Yes	0.000004
Single element = k	1	7	7	Yes	Yes	0.000003
Single element < k	1	7	3	No	Yes	0.000002
All elements > k	3	5	0	No	Yes	0.000005
$k = 0$	3	0	0	Yes	Yes	0.000001
$k = \sum S$	3	20	20	Yes	Yes	0.000005

Table 3: Edge case results for `brute_force_optimization`.

All six cases behave correctly. The $k = 0$ case is worth noting: the empty set (sum = 0) is returned and correctly flagged as an exact match since $0 = k$. When all elements exceed k , the empty set (sum = 0) is returned as the best feasible option.

5.2 Heuristic Algorithm

[This section is authored by Person 2.]

6 Experimental Analysis of the Performance

[This section is authored by Person 2.]

7 Experimental Analysis of the Quality

[This section is authored by Person 3.]

8 Experimental Analysis of the Correctness

[This section is authored by Person 3.]

9 Discussion

[This section is authored by Person 3.]

References

- [1] Karp, R. M. (1972). Reducibility among combinatorial problems. In R. E. Miller and J. W. Thatcher (Eds.), *Complexity of Computer Computations* (pp. 85–103). Plenum Press. https://doi.org/10.1007/978-1-4684-2001-2_9
- [2] Sipser, M. (2012). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning. (Theorem 7.25, p. 297: SUBSET-SUM $\in$ NP. Section 7.5, p. 319: NP-completeness of SUBSET-SUM.)
- [3] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. (Section 35.5: The subset-sum problem.)
- [4] Garey, M. R. and Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman. (Subset Sum is listed as problem SP13.)